

Prompt: Filtro de Fechas en Página de Reservas

Contexto

Actualmente `GET /api/reservations` devuelve todas las reservas de la historia sin ningún filtro. La página muestra todo mezclado. Se necesita:

- **Vista por defecto:** reservas activas (pending/confirmed/checked_in) + futuras, ordenadas por check-in
 - **Selector de fecha** en el encabezado para consultar por rango o fecha puntual
 - Mantener todos los filtros existentes (búsqueda por nombre/hab, filtro por estado)
-

PASO 1 — Backend: agregar parámetros de fecha al endpoint

En `server/routes.ts`, modificar `GET /api/reservations` para aceptar query params opcionales:

```
app.get("/api/reservations", async (req, res) => {
  try {
    const { dateFrom, dateTo, dateMode } = req.query;
    const reservations = await storage.getReservations({
      dateFrom: dateFrom as string | undefined,
      dateTo: dateTo as string | undefined,
      dateMode: dateMode as string | undefined,
    });
    res.json(reservations);
  } catch (error) {
    res.status(500).json({ error: "Error fetching reservations" });
  }
});
```

En `server/storage.ts`, modificar `getReservations` para filtrar por fecha:

```
async getReservations(options?: {
  dateFrom?: string;
```

```

dateTo?: string;
dateMode?: string; // "checkin" | "checkout" | "active"
}): Promise<ReservationWithDetails[]> {

  let query = db.select().from(reservations);

  const conditions = [];

  if (options?.dateFrom || options?.dateTo) {
    // Filtrar por rango de check-in
    if (options.dateFrom) {
      conditions.push(gte(reservations.checkInDate, options.dateFrom));
    }
    if (options.dateTo) {
      conditions.push(lte(reservations.checkInDate, options.dateTo));
    }
  } else {
    // Por defecto: solo reservas activas y futuras (no checked_out, no cancelled
    const today = getArgentinaToday();
    conditions.push(
      or(
        // Reservas activas (cualquier fecha)
        inArray(reservations.status, ["pending", "confirmed", "checked_in"]),
        // Reservas futuras (aunque estén canceladas, para tener contexto)
        and(
          gte(reservations.checkInDate, today),
          ne(reservations.status, "cancelled")
        )
      )
    );
  }

  if (conditions.length > 0) {
    query = query.where(and(...conditions)) as any;
  }

  const allRes = await (query.orderBy(reservations.checkInDate) as any);

  const results: ReservationWithDetails[] = [];
  for (const r of allRes) {
    results.push(await this.enrichReservation(r));
  }
  return results;
}

```

Asegurarse de importar `gte`, `lte`, `inArray`, `or`, `and`, `ne` **de drizzle-orm** si no están

importados ya.

PASO 2 — Frontend: agregar estado y controles de fecha

En `ReservationsPage`, agregar los nuevos estados de fecha:

```
// Agregar junto a los estados existentes
const todayStr = new Date().toLocaleDateString("en-CA", { timeZone: "America/Arge

const [dateMode, setDateMode] = useState<"upcoming" | "today" | "range" | "all">(
const [dateFrom, setDateFrom] = useState(todayStr);
const [dateTo, setDateTo] = useState("");
```

Modificar el query para pasar los parámetros de fecha:

```
const { data: reservations, isLoading } = useQuery<ReservationWithDetails[]>({
  queryKey: ["/api/reservations", dateMode, dateFrom, dateTo],
  queryFn: async () => {
    const params = new URLSearchParams();

    if (dateMode === "today") {
      params.set("dateFrom", todayStr);
      params.set("dateTo", todayStr);
    } else if (dateMode === "range" && dateFrom) {
      params.set("dateFrom", dateFrom);
      if (dateTo) params.set("dateTo", dateTo);
    } else if (dateMode === "all") {
      params.set("dateMode", "all");
    }
    // "upcoming" no manda params → usa el filtro por defecto del backend

    const res = await fetch(`/api/reservations?${params.toString()}`);
    if (!res.ok) throw new Error("Failed to fetch reservations");
    return res.json();
  },
});
```

PASO 3 — Frontend: controles de fecha en el encabezado

Reemplazar el bloque de filtros existente (la Card con el buscador y select de estado) por esta versión ampliada:

```

<Card>
  <CardContent className="p-4">
    <div className="flex flex-col gap-3">

      {/* Fila 1: Selector de modo de fecha */}
      <div className="flex items-center gap-2 flex-wrap">
        <span className="text-sm font-medium text-muted-foreground mr-1">Ver:</sp

      <Button
        variant={dateMode === "upcoming" ? "default" : "outline"}
        size="sm"
        onClick={() => setDateMode("upcoming")}
        data-testid="button-filter-upcoming"
      >
        Activas y futuras
      </Button>

      <Button
        variant={dateMode === "today" ? "default" : "outline"}
        size="sm"
        onClick={() => setDateMode("today")}
        data-testid="button-filter-today"
      >
        Hoy ({todayStr})
      </Button>

      <Button
        variant={dateMode === "range" ? "default" : "outline"}
        size="sm"
        onClick={() => setDateMode("range")}
        data-testid="button-filter-range"
      >
        <CalendarRange className="h-3.5 w-3.5 mr-1.5" />
        Por fechas
      </Button>

      <Button
        variant={dateMode === "all" ? "default" : "outline"}
        size="sm"
        onClick={() => setDateMode("all")}
        data-testid="button-filter-all"
      >
        Todas
      </Button>

      {/* Inputs de rango - solo visible cuando modo = "range" */}

```

```

{dateMode === "range" && (
  <div className="flex items-center gap-2 ml-2">
    <Input
      type="date"
      value={dateFrom}
      onChange={(e) => setDateFrom(e.target.value)}
      className="h-8 w-36 text-sm"
      data-testid="input-date-from"
    />
    <span className="text-muted-foreground text-sm">-</span>
    <Input
      type="date"
      value={dateTo}
      onChange={(e) => setDateTo(e.target.value)}
      className="h-8 w-36 text-sm"
      placeholder="Sin límite"
      data-testid="input-date-to"
    />
  </div>
)}
</div>

{/* Fila 2: Buscador + filtro estado (ya existentes) */}
<div className="flex flex-col gap-4 sm:flex-row sm:items-center">
  <div className="relative flex-1">
    <Search className="absolute left-3 top-1/2 h-4 w-4 -translate-y-1/2 tex
    <Input
      placeholder="Buscar por huésped o habitación..."
      value={searchQuery}
      onChange={(e) => setSearchQuery(e.target.value)}
      className="pl-10"
      data-testid="input-search-reservations"
    />
  </div>
  <Select value={statusFilter} onValueChange={setStatusFilter}>
    <SelectTrigger className="w-[180px]" data-testid="select-filter-status"
      <Filter className="mr-2 h-4 w-4" />
      <SelectValue placeholder="Estado" />
    </SelectTrigger>
    <SelectContent>
      <SelectItem value="all">Todos los estados</SelectItem>
      <SelectItem value="pending">Pendientes</SelectItem>
      <SelectItem value="confirmed">Confirmadas</SelectItem>
      <SelectItem value="checked_in">Check-in</SelectItem>
      <SelectItem value="checked_out">Check-out</SelectItem>
      <SelectItem value="cancelled">Canceladas</SelectItem>
    </SelectContent>
  </div>

```

```

        </Select>
      </div>

    </div>
  </CardContent>
</Card>

```

Agregar `CalendarRange` a los imports de `lucide-react` (buscar la línea que importa los otros iconos y agregarlo).

PASO 4 — Mostrar contador de resultados y ordenamiento

En el encabezado de la tabla (o justo antes de la Card de la tabla), agregar info del total visible:

```

{/* Justo antes del inicio de la tabla */}
{!isLoading && filteredReservations && (
  <div className="flex items-center justify-between text-sm text-muted-foreground" >
    <span>
      {filteredReservations.length} reserva{filteredReservations.length !== 1 ? "
      {dateMode === "today" && " para hoy"}
      {dateMode === "upcoming" && " activas y futuras"}
      {dateMode === "range" && dateFrom && ` desde ${dateFrom}${dateTo ? ` hasta
    </span>
  </div>
)}

```

PASO 5 — Ordenar por check-in en la vista por defecto

La vista "Activas y futuras" debe mostrar las más próximas primero. En el backend ya lo hace con `orderBy(reservations.checkInDate)` , pero en el frontend asegurarse de que `filteredReservations` también esté ordenado:

```

// Después del .filter() existente, agregar sort:
const filteredReservations = reservations
  ?.filter((res) => {
    const guestName = `${res.guest?.firstName} ${res.guest?.lastName}`.toLowerCase()
    const matchesSearch =
      guestName.includes(searchQuery.toLowerCase()) ||
      res.room?.roomNumber?.toLowerCase().includes(searchQuery.toLowerCase());
  })

```

```
const matchesStatus = statusFilter === "all" || res.status === statusFilter;
return matchesSearch && matchesStatus;
})
?.sort((a, b) => {
  // Ordenar por check-in ascendente (más próximo primero)
  return a.checkInDate.localeCompare(b.checkInDate);
});
```

Resumen del comportamiento esperado

Botón	Qué muestra
Activas y futuras (default)	Pending + Confirmed + CheckedIn de cualquier fecha, más futuras no canceladas. Orden: check-in más próximo primero
Hoy	Solo reservas con check-in = hoy
Por fechas	Muestra inputs de rango. Filtra check-in dentro del rango
Todas	Toda la historia, sin filtro de fecha

El filtro de estado y el buscador siguen funcionando sobre lo que muestre el modo de fecha activo.